[PUBLIC]

FastAPI Architecture Guide — NovaTech Internal Platform

Author: Arjun Mehta
Department: Engineering
Last Updated: 2026-04-10
Classification: public

This guide defines the canonical FastAPI architecture adopted by NovaTech's Platform Engineering team as of Q1 2026. It covers project layout, dependency injection patterns, middleware stack, async database access, and integration with our Qdrant-based vector search layer. All new microservices under Project Atlas must conform to this specification.

1. Project Layout

NovaTech standardises on a domain-driven layout for every FastAPI service. The top-level package is split into **routers**, **services**, **repositories**, **schemas**, and **core**. The *core* module hosts settings (via Pydantic BaseSettings), logging configuration, and the lifespan context manager that initialises shared resources such as the async SQLAlchemy engine and the Qdrant AsyncClient.

All route handlers are thin orchestration layers. Business logic lives exclusively in service classes, which are injected via FastAPI's **Depends()** mechanism. This separation has reduced average handler complexity from 47 lines to 12 lines across the four services migrated in Q4 2025 (Gateway, Embedder, Retriever, Reranker).

Layer	Responsibility	Owner
Router	HTTP binding, request validation	Platform Team
Service	Business logic, orchestration	Domain Team
Repository	DB / vector store access	Platform Team
Schema	Pydantic I/O models	Domain Team
Core	Config, logging, lifespan	Platform Team

2. Middleware Stack

The standard middleware order, outermost first, is: **CorrelationIDMiddleware** → **RequestLoggingMiddleware** → **AuthMiddleware** → **RateLimitMiddleware** → **CompressionMiddleware**.

The AuthMiddleware validates JWTs issued by our Keycloak instance and attaches a *RequestContext* object to *request.state*. This context carries the user's role set, tenant ID, and a pre-computed permission bitmap used by the RBAC enforcement layer in Project Atlas's retrieval service.

- CorrelationID is propagated via X-Correlation-ID header to downstream calls.
- Rate limiting uses a Redis sliding-window counter (1 000 rpm per API key by default).
- Response compression is applied for payloads larger than 4 KB (gzip, level 6).
- Auth token verification uses python-jose with RS256; public keys are cached for 5 minutes.

3. Async Database Patterns

All database I/O uses SQLAlchemy 2.0 in fully async mode with asyncpg as the driver. Connection pool size is tuned per-service: the Query Service uses pool_size=20, max_overflow=10, while the Ingest Service uses pool_size=5, max_overflow=2 to avoid overwhelming the write-optimised replica.

A mandatory **get_db()** dependency yields an AsyncSession and commits or rolls back automatically. Direct session access outside of the repository layer is forbidden and enforced via an AST linter rule added in March 2026.

4. Qdrant Integration

The Retriever service connects to Qdrant using the official Python async client (qdrant-client >= 1.9). The QdrantRepository class wraps search, upsert, and delete operations. Hybrid retrieval combines dense vectors (BAAI/bge-base-en-v1.5, 768 dims) with sparse BM25 vectors, fused via Reciprocal Rank Fusion (RRF) with k=60.

Payload filtering enforces RBAC: each point carries a *classification* field ('public', 'internal', 'confidential'). The must filter list is populated at query time from the user's role set. This prevents confidential Finance documents from appearing in results for Engineering-role users.

5. Performance Benchmarks

Endpoint	P50 (ms)	P95 (ms)	P99 (ms)	RPS (sustained)
/ask (RAG)	210	480	820	340
/embed	45	110	190	1 200
/rerank	95	240	410	620
/health	2	5	9	8 000

Benchmarks were run on an AWS m6i.2xlarge with 50 concurrent users using Locust 2.24. The /ask endpoint includes Qdrant search + LLM inference; P95 target of <500 ms is currently breached under peak

load and is tracked under Project Phoenix for Q3 2026 resolution.